

DOCUMENTO DE MODELO Y PLAN

El modelo del núcleo (en simple) y el siguiente hito: Aurora publicable

Núcleo AI-first · Fábrica multi-app

Campo	Valor
Repositorio	https://github.com/hanccotemp/kernel_app
Partes	A) Modelo del núcleo explicado · B) Plan del siguiente hito
Versión	1.0 · Fecha: 2026-06-01
Diagrama gráfico	docs/MODELO_NUCLEO.md (se dibuja en GitHub)

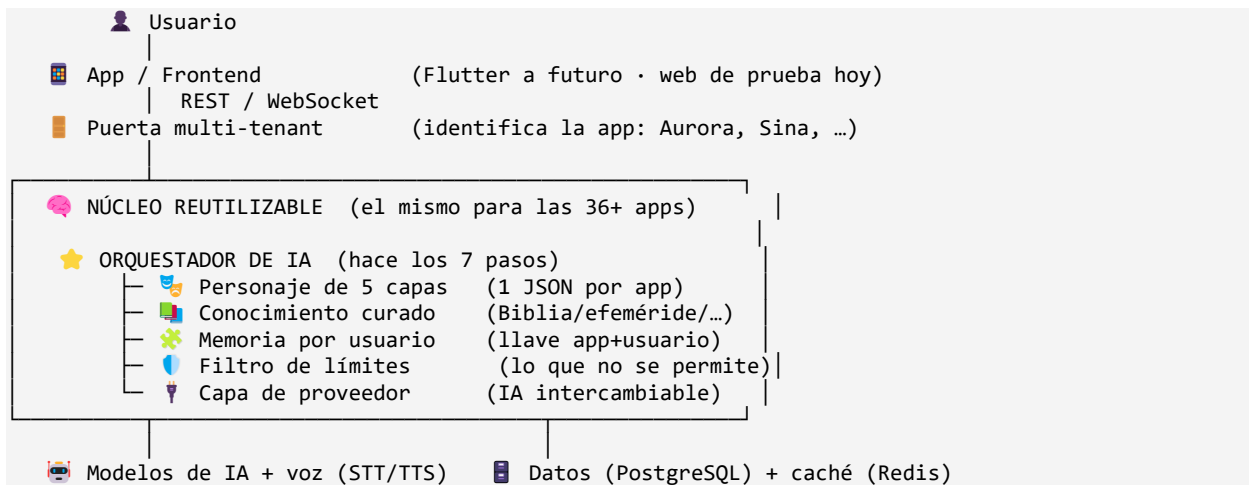
Contenido

PARTE A — El núcleo explicado en simple

Esta parte explica, sin jerga técnica, cómo está construido el motor, para que el dueño del proyecto pueda entenderlo, dar seguimiento y explicarlo a un socio o inversor.

A.1 El diagrama del núcleo

El usuario habla con la app; la app le pasa la pregunta al núcleo; el núcleo (el “cerebro”) la procesa y responde. El mismo núcleo sirve a todas las apps:



Versión gráfica (cajas y flechas): docs/MODELO_NUCLEO.md — se dibuja automáticamente al abrirlo en GitHub.

A.2 Cómo viaja una pregunta: los 7 pasos (ejemplo con Aurora)

Ejemplo: João le escribe a Aurora: “Estoy con mucha ansiedad por el trabajo y no logro dormir.”

1. **Identifica la app y carga su personaje.** El núcleo ve que es Aurora y carga su “forma de ser” (cálida, de fe).
2. **Reúne el contexto del usuario.** Recupera lo que sabe de João (su idioma, su momento).
3. **Inyecta conocimiento curado.** Trae el versículo exacto para “ansiedad” desde la Biblia real (no lo inventa).
4. **Construye el mensaje para la IA.** Junta personaje + contexto + versículo + memoria previa.
5. **Llama a la IA.** La IA redacta una respuesta cálida usando ese versículo.
6. **Aplica los límites.** Revisa que no diga nada prohibido (p. ej., no promete sanaciones).
7. **Entrega la respuesta.** Devuelve el texto (y, si es premium, lo convierte en voz).

Resultado real verificado: Aurora respondió citando Filipenses 4:6 exacto, con una oración para João. Para Sina (astrología) es el mismo recorrido; solo cambian el personaje y la fuente de conocimiento.

A.3 Dónde viven las 5 capas y cómo se agrega una app

El “cerebro” de cada app es un solo archivo de configuración (JSON) con 5 capas. No es programación: es llenar una plantilla.

Capa	Qué define (ejemplo de Aurora)
1 · Identidad	Quién es: nombre, personalidad, tono (cálida, de fe).
2 · Contexto	Qué datos del usuario usa (momento espiritual, idioma).
3 · Conocimiento	De dónde saca los datos duros (la Biblia real).
4 · Límites	Lo que NO puede decir (no promete sanaciones, etc.).
5 · Formato	Cómo responde (breve, con un versículo y una oración).

Agregar una app nueva = soltar un archivo JSON. Verificado: se agregaron 3 apps nuevas (Familia 7) solo con configuración; el motor no se tocó.

A.4 Cómo se guarda y se separa la memoria de cada usuario

Cada usuario, dentro de cada personalidad, tiene su propio “cuaderno” de conversación. La llave que los separa es la combinación (app/personalidad + usuario). Lo de Juan1 con el Religioso jamás aparece en Juan2 ni en otra personalidad. Validado con pruebas (ver Validacion_Memoria_Tecnica.pdf).

A.5 Qué está hecho, a medias y qué falta

Pieza	Estado	Detalle
Orquestador (7 pasos)	✓ Hecho	Probado con IA real (Claude)
Personajes 5 capas	✓ Hecho	Por configuración; 5 apps cargadas
Conocimiento curado	✓ Hecho	Biblia completa + efemérides + corpus genérico
Memoria por usuario	✓ Hecho	Aislada por (app+usuario); falta persistirla
Multi-tenant	✓ Hecho	Mismo motor, varias apps aisladas
Multimodal (imagen)	✓ Hecho	Capacidad reutilizable; probada con Claude
Capa de proveedor IA	✓ Hecho	mock/deepseek/openai/anthropic intercambiables
Caché	● A medias	En memoria; falta Redis real
Base de datos	● A medias	En memoria; falta PostgreSQL real
Voz (STT/TTS)	● A medias	Estructura lista; falta proveedor real
Frontend	● Mínimo	Ciente web de prueba; falta app Flutter
Pagos / Auth	● Andamiaje	Falta RevenueCat + login real
Infraestructura	✗ Falta	Falta desplegar en AWS São Paulo

A.6 El modelo de datos (entidades y relaciones)

Los datos son genéricos (no atados a “astrología” ni “fe”), para servir a muchas apps. Entidades principales:

Entidad	Qué guarda	Se relaciona con
App	Cada app/personalidad (Aurora, Sina...)	Config, Usuarios, Roles
Config	El personaje 5 capas, precios, piel, idiomas	App (1 a 1)
Usuario	Datos del usuario final y su perfil	App, Suscripción, Conversación
Suscripción	Plan free/premium y add-ons (voz)	Usuario, App
Conversación	El hilo de chat de un usuario en una app	Usuario+App, Mensajes
Mensaje	Cada turno (usuario o IA)	Conversación

Entidad	Qué guarda	Se relaciona con
Recurso	Contenido abstracto (versículo, tránsito...)	App
Rol / Permiso	Quién puede hacer qué en el back-office	App, Usuario (staff)

Diagrama entidad-relación completo (gráfico): docs/MODELO_DATOS.md.

PARTE B — Siguiente hito: Aurora publicable

El núcleo (el cerebro) está probado. El siguiente salto es darle “cuerpo”: una app real que una persona pueda descargar, usar, suscribirse y pagar. A continuación, las respuestas a las 6 preguntas.

Nota sobre las estimaciones: plazos y horas son estimaciones de ingeniería con supuestos explícitos; los montos en dinero de desarrollo dependen de la tarifa del equipo (CHA/DEAN), por lo que se entregan en horas para que multipliquen por su tarifa. Los costos de servicios sí se estiman.

B.1 Los 6 entregables

- 1. Persistencia real (PostgreSQL): que usuarios, memoria y suscripciones sobrevivan reinicios.
- 2. Frontend Flutter: la app publicable de Aurora (iOS + Android).
- 3. Voz real (STT + TTS): entrada por voz y respuesta hablada (add-on premium).
- 4. Pagos reales (RevenueCat + compras in-app): free, base y add-on de voz, prueba 7 días.
- 5. Autenticación real: login email / Google / Apple.
- 6. Infraestructura: AWS São Paulo con auto-escalado + Redis real.

B.2 (Pregunta 3) Orden recomendado y por qué

8. **Persistencia (PostgreSQL) — primero.** Base de todo; sin esto se pierden datos. Riesgo bajo, alto valor.
9. **Auth + esqueleto Flutter — en paralelo.** La app necesita login desde el inicio; conviene definirlo antes de construir pantallas para no rehacerlas.
10. **Pagos (RevenueCat).** Una vez hay usuarios y pantallas, se conecta el cobro (free/base/voz + prueba 7 días).
11. **Voz real (STT/TTS) — puede ir en paralelo.** Es el diferenciador; entra por la capa de voz que ya existe, sin frenar lo demás.
12. **Infraestructura AWS + Redis — al final.** Para publicar con escala; se hace cuando la app ya funciona de extremo a extremo.

Coincide con la propuesta del solicitante, con un ajuste: adelantar la autenticación para que el frontend no se rehaga.

B.3 (Pregunta 1) Plazo estimado

Supuesto: 1 desarrollador full-stack con experiencia en Flutter, dedicado (con un segundo en paralelo, los tiempos bajan ~30-40%).

Entregable	Horas aprox.	Calendario (1 dev)
Persistencia PostgreSQL	30 – 50 h	~1 semana
Frontend Flutter (MVP Aurora)	120 – 200 h	~4 – 6 semanas
Auth (email/Google/Apple)	30 – 50 h	~1 – 1.5 semanas
Pagos (RevenueCat + IAP)	40 – 60 h	~1.5 – 2 semanas
Voz real (STT + TTS)	40 – 60 h	~1.5 – 2 semanas
Infra AWS São Paulo + Redis	40 – 70 h	~1.5 – 2 semanas

Entregable	Horas aprox.	Calendario (1 dev)
Pruebas, pulido y envío a tiendas	40 – 60 h	~1 – 2 semanas

Total a app publicada: ≈ 10 – 14 semanas (2.5 – 3.5 meses) con 1 dev dedicado y solapamientos; ≈ 6 – 9 semanas con 2 devs en paralelo. A esto se suma la revisión de la tienda (Apple suele tardar de días a 1–2 semanas).

B.4 (Pregunta 2) Costo

Desarrollo (una vez): ≈ 340 – 550 horas en total (ver tabla anterior). El monto = horas × tarifa del equipo. Recomendación: pedir cotización a CHA/DEAN con este desglose por entregable, y construir por etapas para validar antes de invertir en todo.

Servicios recurrentes (mensual, a baja escala ~1.000 usuarios):

Servicio	Costo aprox.
AWS São Paulo (contenedor + PostgreSQL + Redis chicos)	≈ US\$ 80 – 250 / mes
IA (DeepSeek, con caché)	≈ US\$ 10 – 50 / mes a baja escala
Voz STT + TTS (pago por uso, add-on premium)	≈ US\$ 0.015 – 0.02 por interacción de voz
RevenueCat (pagos)	Gratis hasta US\$ 2,500/mes de ingresos; luego ~1%
Apple Developer / Google Play	US\$ 99 / año · US\$ 25 (único)

Conviene por etapas: sí. La etapa 1 (Persistencia + Flutter + Auth + Pagos, sin voz ni AWS gestionado) ya permite una versión usable; voz e infraestructura escalable se suman después.

B.5 (Pregunta 4) Cómo conectaremos la voz

La voz entra por la “capa de voz” que ya existe en el núcleo (hoy es un stub). Como la capa de proveedor de IA, será intercambiable, para no quedar atados a un proveedor y proteger el margen.

Función	Proveedor propuesto	Costo aprox.
STT (voz → texto)	Deepgram (Nova) — multilingüe ES/PT/EN	≈ US\$ 0.004 – 0.006 / min
TTS (texto → voz)	Google/Azure Neural (costo) o ElevenLabs (premium)	≈ US\$ 16 / millón de caracteres

Costo combinado: ≈ US\$ 0.015 – 0.02 por interacción de voz. Por eso la voz se ofrece como add-on premium (protege el margen). Se empieza con una opción económica y se puede subir a ElevenLabs para voces más naturales sin cambiar el resto.

B.6 (Pregunta 5) Qué necesitamos de su parte

- Cuentas de tienda a nombre de la empresa: Apple Developer (US\$99/año) y Google Play (US\$25).
- Accesos / cuentas: AWS (o autorización para crearla), claves de IA (DeepSeek), de voz (Deepgram/Google), y cuenta RevenueCat.
- Contenido de Aurora validado: plan de lectura, “momentos”, textos. (La versión bíblica ya está resuelta en dominio público; si quieren RVR1960, requiere licencia.)
- Decisiones de negocio: precios finales por mercado/moneda, branding (logo, colores) de Aurora, e idiomas de lanzamiento.
- Legal: política de privacidad y términos (requeridos por las tiendas y por LGPD).

B.7 (Pregunta 6) Confirmación: no se reescribe el núcleo

Confirmado. Cada entregable entra por una interfaz que ya quedó preparada; el orquestador y su lógica no cambian:

Entregable	Entra por la interfaz ya existente
PostgreSQL	Reemplaza la capa de datos (mismos find/insert/remove)
Redis	Reemplaza la caché (mismos get/set/wrap)
Voz real	Rellena la capa de voz (stub) + STT en la puerta de entrada
Pagos	Reemplaza el control de suscripción (gating ya existe)
Auth	Reemplaza el andamiaje de login por JWT real
Flutter	Es otro cliente del MISMO API REST/WebSocket
AWS	Despliegue; no cambia el código

Conclusión

Parte A: el modelo del núcleo queda explicado en simple, con diagrama, los 7 pasos, las 5 capas, la memoria, el estado y el modelo de datos. Parte B: el hito “Aurora publicable” es alcanzable en ~2.5–3.5 meses (1 dev) sin reescribir el núcleo, entrando todo por las interfaces ya preparadas.

Primer paso ejecutable de inmediato: la Persistencia (PostgreSQL), que además cierra la única nota pendiente de la validación de memoria.